Umm Al-Qura University
College of Computing
Department of Computer Science and Artificial Intelligence

Cinema Ticket Booking System

Database - DS2201T

Team Members:

Sara Akram Bashawri 444003064

Lamar Ashraf Andragiri 444006424

Yara Ismail Alsaimy 444007214

Project leader:

Farah Abdullah Alhazmi 444002659

lecture (theoretical) group: 9

Instructor's Name: Dr. Ruba A. Aljamal

Table of Contents

SECTION 1 (CONCEPTUAL AND LOGICAL DESIGN): 5

SYSTEM OVERVIEW 5
IDENTIFY ENTITY 5
ER DIAGRAM 5
RELATIONAL SCHEMA 6

SECTION 2 (IMPLEMENTATION): 7

.1 CREATE DATABASE: 7

.2 CREATE TABLES: 7

2.1 CREATE MOVIE TABLE 7
2.2 CREATE SCREENING TABLE 8
2.3 CREATE CUSTOMER TABLE 9
2.4 CREATE TICKET TABLE 10

3. INSERT DATA INTO TABLES: 11

3.1 INSERT DATA INTO MOVIE 11
3.2 INSERT DATA INTO SCREENING 11
3.3 INSERT DATA INTO CUSTOMER 12
3.4 INSERT DATA INTO TICKET 12

SECTION 3 (SQL QUERIES IMPLEMENTATION AND TESTING): 14

1. BASIC SELECT QUERY 14

1.1 RETRIEVE MOVIES TITLE WITH AGE RATING PG 14
1.2 RETRIEVE LOW-PRICE SCREENINGS 14

2. AGGREGATE FUNCTION QUERY 15

2.1 CALCULATE AVERAGE TICKET PRICE 15
2.2 RETRIEVE LOWEST AND HIGHEST TICKET PRICES 16

3. JOIN QUERY 16

3.1 DISPLAY TICKET, CUSTOMER, AND MOVIE INFORMATION..... 16

4. SUBQUERY 17

4.1 IDENTIFY CUSTOMERS WHO PURCHASED THE HIGHEST-PRICED SCREENING 17

5. VIEW CREATION 18

5.1 CREATE A TICKET REPORT VIEW (V_TICKET_REPORT) 18

Task Distribution:

Description	Created By
Movie table + its part in ER & Schema	**Yara**
Screening table + its part in ER & Schema	**Farah**
Customer table + its part in ER & Schema	**Lamar**
Ticket table + its part in ER & Schema	**Sara**
SELECT+ Aggregate Functions + Joins+ Subquery to answer a specific question+ View	**All members**

Section 1 (Conceptual and Logical Design):

System Overview

This database system manages the booking of cinema tickets.

It stores information about movies, their screenings, customers, and ticket reservations.

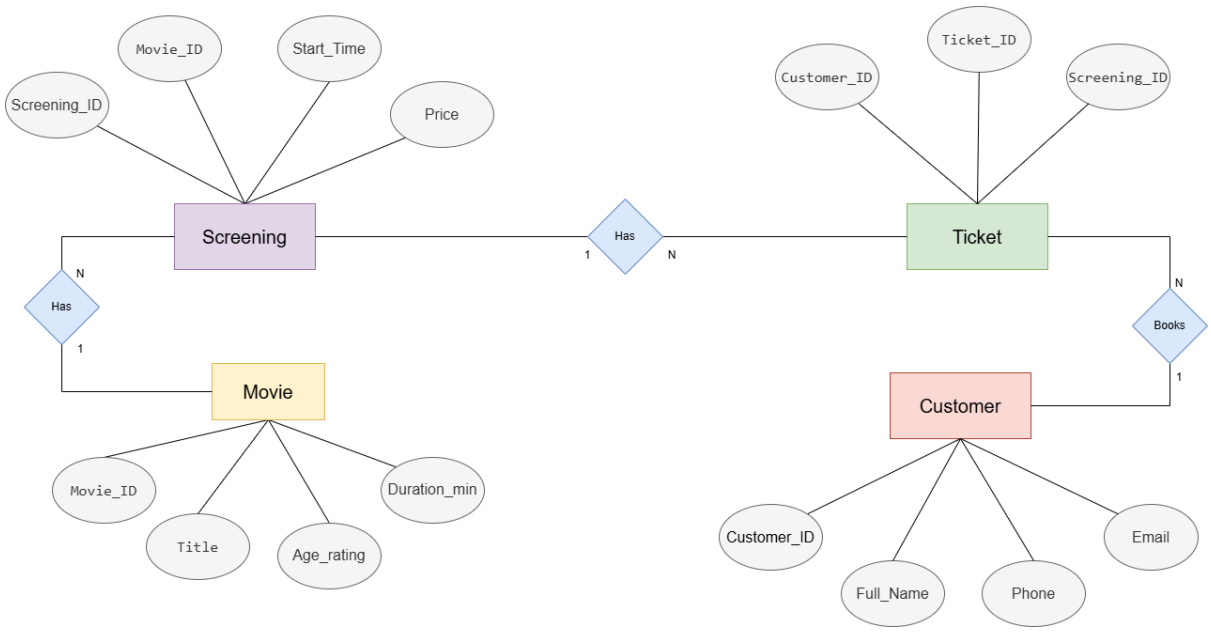
Identify Entity

- A Movie is identified by its (Movie_ID, Title, Duration_min, Age_rating).
- A Screening is identified by its (Screening_ID, Movie_ID, Start_time, price).
- A Customer is identified by their (Customer_id, Full_name, Phone, Email).
- A Ticket is identified by its (Ticket_ID, Screening_ID, Customer_ID).

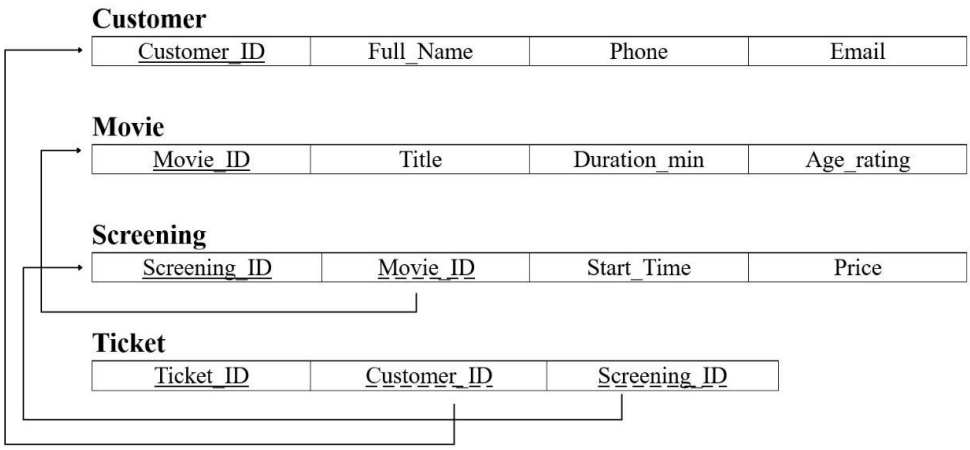
Each movie can have multiple screenings, and each screening can have many tickets.

A customer can book multiple tickets, but each ticket is linked to one customer and one screening.

ER Diagram



Relational Schema



Section 2 (Implementation):

1. Create DataBase:

firstlab-ds | Cinema-Booking-DB

```sql
CREATE DATABASE Cinema_Booking;
USE Cinema_Booking;
```

Description:

Creates a new database named **Cinema_Booking** and sets it as the active database for the following operations.

2. Create Tables:

2.1 Create Movie Table

```sql
-- Movie
Create table Movie (
Movie_ID INT PRIMARY KEY,
Title VARCHAR(100) NOT NULL,
Duration_min INT NOT NULL,
Age_rating VARCHAR(10)  NOT NULL,
CHECK (Duration_min > 0)
);
DESCRIBE Movie;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Movie_ID	int	NO	PRI	NULL	
Title	varchar(100)	NO		NULL	
Duration_min	int	NO		NULL	
Age_rating	varchar(10)	NO		NULL	

Description:

The "Movie" table is created to store information about movies, including:

Movie_ID: The unique identifier for the movie (Primary Key).

Title: The title of the movie.

Duration_min: The duration of the movie in minutes.

Age_rating: The age rating of the movie (e.g., G, PG).

The CHECK constraint ensures that the duration is greater than zero.

DESCRIBE is used to display the table structure after creation

2.2 Create Screening Table

```sql
-- Screening
CREATE TABLE Screening (
  Screening_ID INT PRIMARY KEY,
  Movie_ID INT NOT NULL,
  Start_time DATETIME NOT NULL,
  Price DECIMAL(6,2) NOT NULL,
  FOREIGN KEY (Movie_ID) REFERENCES Movie(Movie_ID)
  );
DESCRIBE Screening;
```

Output:

Field	Type	Null	Key	Default	Extra
Screening_ID	int	NO	PRI	NULL	
Movie_ID	int	NO	MUL	NULL	
Start_time	datetime	NO		NULL	
price	decimal(6,2)	NO		NULL	

Description:

The "Screening" table is created to store screening details, such as:

Screening_ID: The unique identifier for the screening (Primary Key).

Movie_ID: The movie's ID, linked to the Movie table by a foreign key.

Start_time: The start date and time of the screening.

Price: The ticket price for the screening

2.3 Create Customer Table

```sql
-- Customer
CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY,
    Full_Name VARCHAR(50) NOT NULL,
    Phone VARCHAR(10) UNIQUE NOT NULL,
    Email VARCHAR(50) UNIQUE NOT NULL
);
DESCRIBE Customer;
```

Output:

Field	Type	Null	Key	Default	Extra
Customer_ID	int	NO	PRI	NULL	
Full_Name	varchar(50)	NO		NULL	
Phone	varchar(10)	NO	UNI	NULL	
Email	varchar(50)	NO	UNI	NULL	

Description:

The "Customer" table stores information about customers, including:

Customer_ID: The unique identifier for the customer (Primary Key).

Full_Name: The full name of the customer.

Phone: The customer's phone number (must be unique).

Email: The customer's email address (must be unique).

2.4 Create Ticket Table

```sql
-- Ticket
CREATE TABLE Ticket (
  ticket_id    INT PRIMARY KEY,
  screening_id INT NOT NULL,
  customer_id  INT NOT NULL,
  CONSTRAINT fk_ticket_screening
    FOREIGN KEY (screening_id) REFERENCES Screening(screening_id),
  CONSTRAINT fk_ticket_customer
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);
DESCRIBE Ticket;
```

Output:

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
ticket_id	int	NO	PRI	NULL	
screening_id	int	NO	MUL	NULL	
customer_id	int	NO	MUL	NULL	

Description:

The "Ticket" table stores ticket information, including:

ticket_id: The unique identifier for each ticket (Primary Key).

screening_id: The ID of the screening (linked to the Screening table).

customer_id: The ID of the customer who purchased the ticket (linked to the Customer table).

Constraints:

Primary Key: ticket_id

Foreign Keys:

- screening_id references Screening(screening_id)
- customer_id references Customer(customer_id)

This table links each ticket to its related screening and customer.

3. Insert Data into Tables:

3.1 Insert Data into Movie

```sql
-- Insert data into Movie
INSERT INTO Movie (Movie_ID, Title, Duration_min, Age_Rating) VALUES
(1, 'Beauty and the Beast', 84, 'G'),
(2, 'Coco', 105, 'PG'),
(3, 'Cars', 117, 'G'),
(4, 'Lilo & Stitch', 85, 'PG'),
(5, 'The Lion King', 89, 'G');
SELECT * FROM Movie;
```

Output:

Result Grid | Filter Rows: | Export:

Movie_ID	Title	Age_rating	duration_min
1	Beauty and the Beast	G	84
2	Coco	PG	105
3	Cars	G	117
4	Lilo & Stitch	PG	85
5	The Lion King	G	89

3.2 Insert Data into Screening

```sql
-- Insert data into Screening
INSERT INTO Screening (Screening_ID, Movie_ID, Start_time, Price) VALUES
(101, 1, '2025-01-10 18:00:00', 45.00),
(102, 2, '2025-01-10 21:00:00', 50.00),
(103, 3, '2025-01-11 17:30:00', 40.00),
(104, 4, '2025-01-11 19:45:00', 35.00),
(105, 5, '2025-01-12 20:15:00', 42.00);
SELECT * FROM Screening;
```

Output:

Screening_ID	Movie_ID	Start_time	price
101	1	2025-01-10 18:00:00	45.00
102	2	2025-01-10 21:00:00	50.00
103	3	2025-01-11 17:30:00	40.00
104	4	2025-01-11 19:45:00	35.00
105	5	2025-01-12 20:15:00	42.00

Screening 28

3.3 Insert Data into Customer

```sql
-- Insert data into Customer
INSERT INTO Customer (Customer_ID, Full_Name, Phone, Email) VALUES
(1, 'Ruba Ahmed Aljmal', '0505971048', 'ruba.ahmed@gmail.com'),
(2, 'Sara Akram Bashawri', '0504838877', 'sara.akram@gmail.com'),
(3, 'Lamar Ashraf Andragiri', '0508264075', 'lamar.ashraf@gmail.com'),
(4, 'Yara Ismail Alsaimy', '0502753335', 'yara.ismail@gmail.com'),
(5, 'Farah Abdullah Alhazmi', '0507654889', 'farah.abdullah@gmail.com');
SELECT * FROM Customer;
```

Output:

Customer_ID	Full_Name	Phone	Email
1	Ruba Ahmed Aljmal	0505971048	ruba.ahmed@gmail.com
2	Sara Akram Bashawri	0504838877	sara.akram@gmail.com
3	Lamar Ashraf Andragiri	0508264075	lamar.ashraf@gmail.com
4	Yara Ismail Alsaimy	0502753335	yara.ismail@gmail.com
5	Farah Abdullah Alhazmi	0507654889	farah.abdullah@gmail.com

3.4 Insert Data into Ticket

```sql

-- Insert data into Ticket
INSERT INTO Ticket (ticket_id, screening_id, customer_id) VALUES
(201, 101, 2),
(202, 102, 3),
(203, 103, 4),
(204, 104, 5),
(205, 105, 1);
SELECT * FROM Ticket;
```

Output:

ticket_id	screening_id	customer_id
201	101	2
202	102	3
203	103	4
204	104	5
205	105	1

Section 3 (SQL Queries Implementation and Testing):

1. Basic SELECT Query

1.1 Retrieve Movies Title with Age Rating PG

Description:

This query retrieves movies from the Movie table where the Age_rating is PG. It displays the movie's title along with its age rating.

Purpose:

This helps filter movies based on their suitability to identify family-friendly movies suitable for children and general audiences

```
SELECT Title, Age_rating
FROM Movie
WHERE Age_rating = 'PG';
```

Output:

	Title	Age_rating
▶	Coco	PG
	Lilo & Stitch	PG

1.2 Retrieve Low-Price Screenings

Description:

This query retrieves screenings from the Screening table where the Price is less than 45.
It displays the screening ID, movie ID, start time, and the ticket price.

Purpose:

This helps identify screenings that offer lower-priced tickets, making it easier to find affordable movie options.

```sql
SELECT Screening_ID, Movie_ID, Start_time, Price
FROM Screening
WHERE Price < 45;
```

Output:

Screening_ID	Movie_ID	Start_time	Price
103	3	2025-01-11 17:30:00	40.00
104	4	2025-01-11 19:45:00	35.00
105	5	2025-01-12 20:15:00	42.00
NULL	NULL	NULL	NULL

2. Aggregate Function Query

2.1 Calculate Average Ticket Price

Description:

This query calculates the average value of the Price column from the *Screening* table.
It displays a single result showing the overall average ticket price.

Purpose:

This helps determine the general pricing level of screenings and understand the typical cost for movie tickets.

```sql
SELECT AVG(Price) AS Average_Price
FROM Screening;
```

Output:

Average_Price
42.400000

2.2 Retrieve Lowest and Highest Ticket Prices

Description:

This query retrieves the minimum and maximum values from the *Price* column in the Screening table.
It displays the lowest ticket price and the highest ticket price available.

Purpose:

This helps identify the price range of screenings, giving a clear idea of the cheapest and most expensive ticket options.

```
SELECT MIN(Price) AS Lowest_Price,
MAX(Price) AS Highest_Price
FROM Screening;
```

Output:

	Lowest_Price	Highest_Price
▶	35.00	50.00

3. JOIN Query

3.1 Display Ticket, Customer, and Movie Information

Description:

This query retrieves ticket information by joining the Ticket**,** Customer**,** Screening, and Movie tables.
It displays the ticket ID, the customer's full name, and the movie title associated with each ticket.

Purpose:

This helps combine data from multiple tables to show which customer purchased which ticket and for which movie, giving a complete view of ticket records.

```sql
SELECT
    t.ticket_id,
    c.Full_Name,
    m.Title
FROM Ticket t
JOIN Customer c ON t.customer_id = c.Customer_ID
JOIN Screening s ON t.screening_id = s.Screening_ID
JOIN Movie m ON s.Movie_ID = m.Movie_ID;
```

Output:

ticket_id	Full_Name	Title
201	Sara Akram Bashawri	Beauty and the Beast
202	Lamar Ashraf Andragiri	Coco
203	Yara Ismail Alsaimy	Cars
204	Farah Abdullah Alhazmi	Lilo & Stitch
205	Ruba Ahmed Aljmal	The Lion King

4. Subquery

4.1 Identify Customers Who Purchased the Highest-Priced Screening

Description:

This query retrieves the full names of customers who purchased a ticket for the screening with the highest price.

It uses nested subqueries to first find the screening with the maximum price, then the customers who bought tickets for that screening.

Purpose:

This helps identify which customers paid for the most expensive screening, useful for analyzing premium ticket buyers or high-value customers.

```sql
SELECT Full_Name
FROM Customer
WHERE Customer_ID IN (
    SELECT customer_id
    FROM Ticket
    WHERE screening_id = (
        SELECT Screening_ID
        FROM Screening
        WHERE Price = (SELECT MAX(Price) FROM Screening)
    )
);
```

Output:

	Full_Name
▶	Lamar Ashraf Andragiri

5. View Creation

5.1 Create a Ticket Report View (v_ticket_report)

Description:

This query creates a view named v_ticket_report that combines data from the Ticket, Customer, Screening, and Movie tables.

It displays each ticket along with the customer's name, the movie title, the screening time, and the ticket price.

Purpose:

This helps generate a clean, ready-to-use report that simplifies accessing ticket details without rewriting multiple JOIN statements.

```sql
DROP VIEW IF EXISTS v_ticket_report;
CREATE VIEW v_ticket_report AS
SELECT
    t.ticket_id,
    c.Full_Name       AS customer_name,
    m.Title           AS movie_title,
    s.Start_time,
    s.Price
FROM Ticket t
JOIN Customer  c ON t.customer_id  = c.Customer_ID
JOIN Screening s ON t.screening_id = s.Screening_ID
JOIN Movie m ON s.Movie_ID = m.Movie_ID;

SELECT *
FROM v_ticket_report
ORDER BY Start_time, ticket_id;
```

Output:

ticket_id	customer_name	movie_title	Start_time	Price
201	Sara Akram Bashawri	Beauty and the Beast	2025-01-10 18:00:00	45.00
202	Lamar Ashraf Andragiri	Coco	2025-01-10 21:00:00	50.00
203	Yara Ismail Alsaimy	Cars	2025-01-11 17:30:00	40.00
204	Farah Abdullah Alhazmi	Lilo & Stitch	2025-01-11 19:45:00	35.00
205	Ruba Ahmed Aljmal	The Lion King	2025-01-12 20:15:00	42.00